Full Stack Development / Further Web Programming

Semester 1, 2026

Dipto Pratyaksa (dipto.pratyaksa@rmit.edu.au)

Week 04

useEffect hook, custom hooks,

routing,

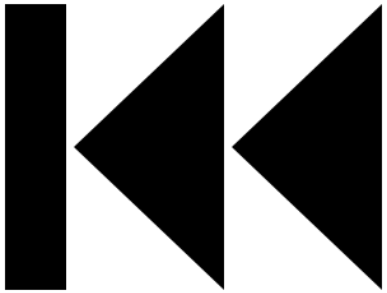
interaction between components and form handling

RMIT UNIVERSITY

CRICOS provider number: 00122A | RTO Code: 3046

Before we start

Previously..



Segment 1

Hooks

- useEffect *again*
- Writing your own (custom)
- React Developer Tools (RDT)

useEffect *hook*

- The Effect Hook lets you perform side effects in function components.
- *Side-effects 'eh!!. .do I have to learn something new?*
  - *"There won't be much to learn. In fact, we'll spend most of our time unlearning." - Dan Abramov*
- What are effects?
- Examples
  - Fetching data
  - Reading from local storage Registering and deregistering event listeners
  - Manually changing DOM elements …

Effects

- Effects are always executed after render
- Hooks documentation by Facebook states-
  - Mutations, subscriptions, timers, logging, and other side effects are not allowed inside the main body of a function component (referred to as React's render phase).
  - Doing so will lead to confusing bugs and inconsistencies in the UI.
  - Instead, use *useEffect*.
    - The function passed to useEffect will run after the render is committed to the screen.
- To opt out or skip effects, you have to understand basic JavaScript concepts about values

useEffect *recap*

- useEffect runs after every render (*by default*), and can optionally clean up for itself before it runs again.
- If you want your effects to run less often, you can provide a second argument – an array of values. Think of them as the *dependencies* for that effect.
  - If one of the dependencies has changed since the last time, the effect will run again.
- When you call useEffect in your component, this is effectively queuing or scheduling an effect to maybe run, after the render is done.
- After rendering finishes, useEffect will check the list of dependency values against the values from the last render, and will call your effect function if any one of them has changed.

useEffect *on props, state change*

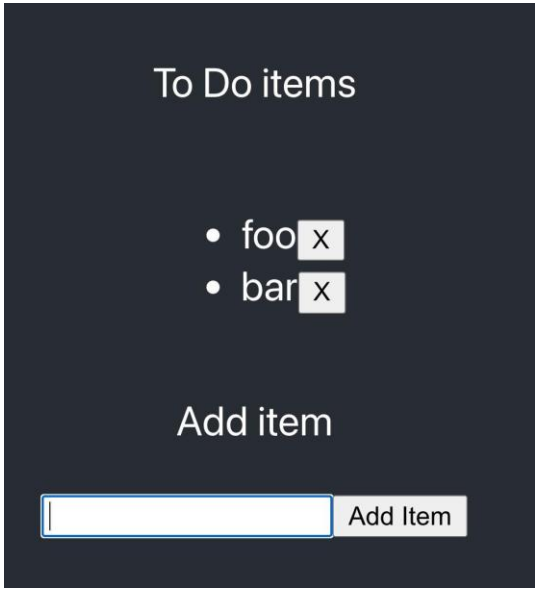
- By default, useEffect runs after every render, but it's also perfect for running some code in response to a state change.
  - You can limit when the effect runs by passing the second argument to useEffect.
- Second argument as an array of "dependencies" – variables that, if changed, the effect should rerun. These can be any kind of variable: props, state, or anything else.
- Time to look at examples:
  - example1 (an example to show when useEffect runs)
  - example2 (prop change)

Lectorial Exercise

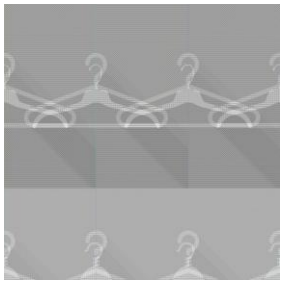
- Do you foresee any issues with useEffect hook? *When can its use be problematic?*

useEffect *on fetching data*

- We will now look at an example of data fetching from localStorage and with *useEffect* in the picture.
- The web app allows use to add to-do items in a list
- These items are stored in a localStorage
- The web app allows use to remove the items
- example3
  - check the comment entries



Custom *hook*



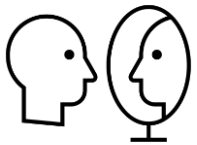
- You could even write your own (*customised*) hook.
- But why!!
- Custom hooks are a handy way to encapsulate hook-related logic that can be re-used across components.
- Think of a custom hook as a super-powered helper function.
- You can call hooks inside custom hooks!
- Custom hooks are-
  - functions whose names begin with 'use...'
  - functions which can call other hooks

Custom *hook*

- We saw one example in the previous week
- But we will keep revisiting the custom hooks
- example4

React Developer Tools: *Chrome*

- This browser extension is available for Chrome and Firefox and is designed to help you debug React applications.
- *Open Extensions page in Chrome*
  - *Search for React Developer Tools*
  - *Install and Enable*
- Here is a good page to learn about these tools
  - *https://react-devtools-tutorial.vercel.app/*



Lectorial Exercise

- Can you name few scenarios where you can justify writing a custom hook? Explain your answer.

Rules of hooks

This PhotoThis Photo by Unknown Author is licensed under CC BY-SA

- Outlined on documentation page:
  https://reactjs.org/docs/hooks-rules.html
  1. Only Call Hooks at the Top Level
  2. Only Call Hooks from React Functions
- Facebook recommends
  - *ESLint plugin called eslint-plugin-react-hooks that enforces these two rules. You can add this plugin to your project if you'd like to try it*

Segment 2

Routing

Interaction between functional components

useContext

Routing

- We've talked about the basic Page Routing a little bit already
- It provides file system-based routing
- But how do we actually navigate
- This week we will look at two ways of navigating:
  - Link component – example5
  - useRouter hook – example6

Ref: You can read about these at: https://nextjs.org/docs/pages/building-your-application/routing

Interaction between components

- Your page will often have multiple components and sometimes
  1. Parent component will need to interact with Child component
  2. Child component will need to interact with Parent component
  3. Children components will need to interact with each other (*siblings*)

Parent - Child

- We have already seen this- the simplest form of communication between components is via properties —*props*.

```
function App(){
  return <div>
    <AppChild name="JabbaTheHut" />
    </div>
}

function AppChild(props){
  return <span>
      My name is {props.name}
    </span>
}
```

- Props are the parameters passed into child components by parents, similar to arguments to a function.
- You can <u>pass events</u> from parent-child too
- example7

Child – Parent

- This is *trickier!*
- When a child needs to update the parent as to changes, they cannot just modify properties. Children cannot update props.
- A parent can pass in a functional prop, and the child can call that function.

```
function App(){
  const [name, setName] = React.useState("JabbaTheHut");
  return <div>
      <AppChild name={name} onChangeName={()=>{setName("JabbaTheHutt")}}/>
    </div>
}

function AppChild(props){
  return <span>
      My name is {props.name}
      <button onClick={props.onChangeName}>Change Name</button>
    </span>
}
```

Child – Parent

- It often happens that child components need to pass arguments up along with their events.
- This can be achieved by adding arguments to the functional prop *callback*
- Time for an example
- example8

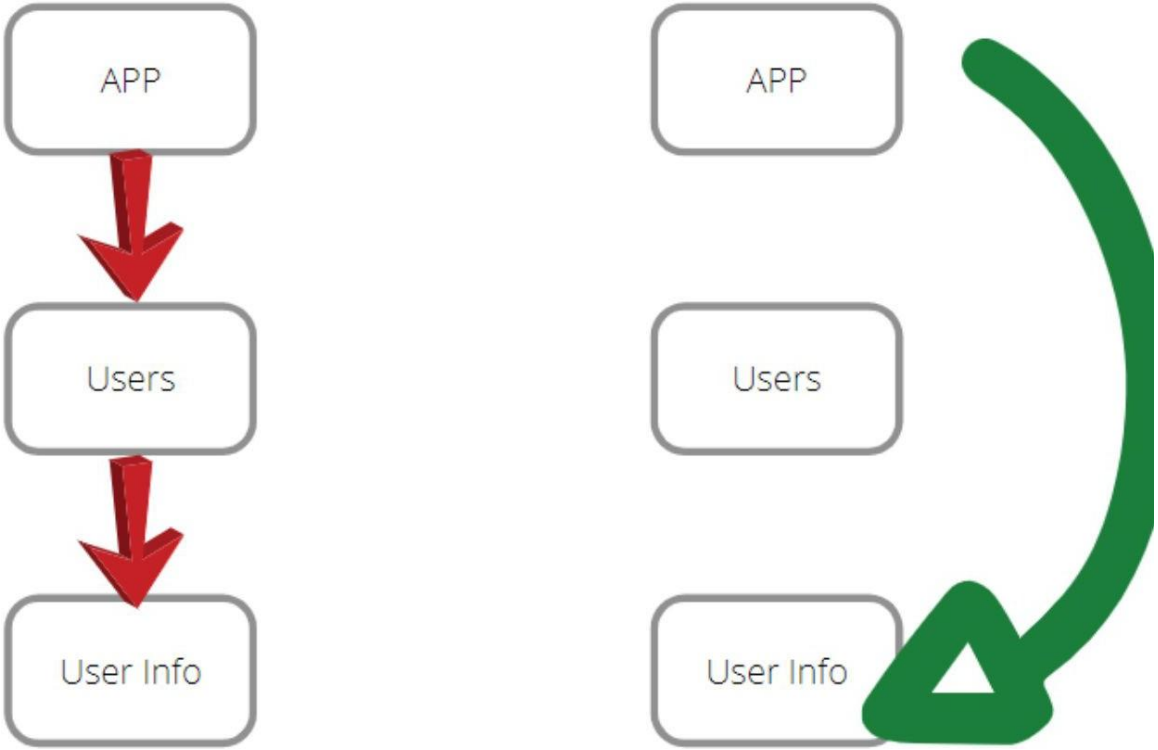
Child - Child

- How can siblings communicate?
- How will a child component communicate with other child component?
- *This is not as easy as others*
  - *We can either use a combination of props and callback*
  - *Use context hook*

useContext

- In a typical React application, data is passed top-down (parent to child) via props, but such usage can be cumbersome for certain types of props
- Sometimes multiple components need the information
- When information/data that needs to be passed becomes complex, the use of props becomes frustrating
- Context provides a way to share values like these between components without having to explicitly pass a prop through every level of the tree
- Context is designed to share data that can be considered "*global*" for a tree of React components, such as the current authenticated user, theme, or preferred language.

Without & With context!



useContext

- Context is primarily used when some data needs to be accessible by many components at different nesting levels.
  - Apply it carefully because it makes component reuse more difficult.
- `const MyContext =`

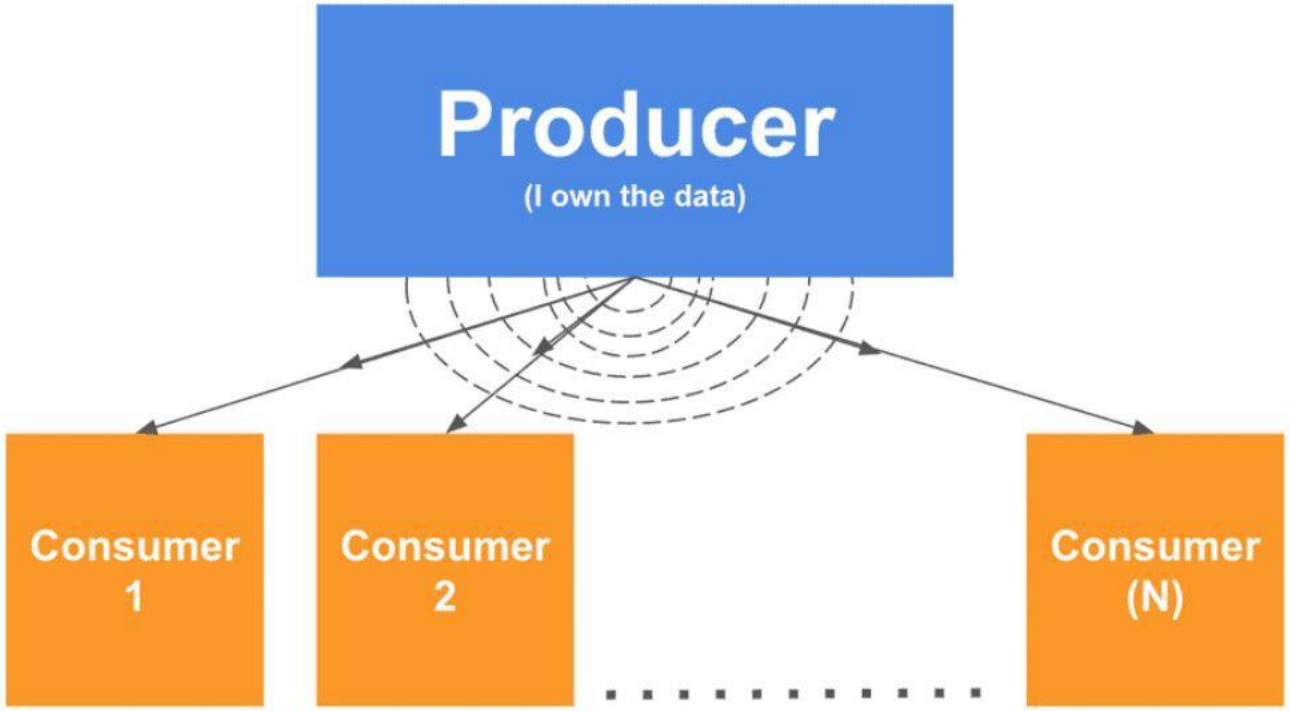
  `React.createContext(defaultValue);`
  - Creates a Context object.
  - When React renders a component that subscribes to this Context object it will read the current context value from the closest matching Provider above it in the tree.
  - *What is a provider?*

useContext

- Every Context object comes with a Provider React component that allows consuming components to subscribe to context changes.
- The Provider component accepts a value prop to be passed to consuming components that are descendants of this Provider. One Provider can be connected to many consumers.
  - `<MyContext.Provider value={/* some value */}>`
- All consumers that are descendants of a Provider will re-render whenever the Provider's value prop changes.
  - *Now, there is a consumer!!*

This Photo by Unknown Author is licensed under CC BY-SA-NC

useContext *producer* and *consumer*



Reference: https://content.breatheco.de/en/lesson/context-api

useContext

- A React component that subscribes to context changes.

```
<MyContext.Consumer>
    {value => /* render something based on the context value */}
</MyContext.Consumer>
```

- Time to now look at an example
  - example9

- We are not yet done with Context- we will revisit it next week!

References

- Reference: *The road to react (2021 edition), by Robin Weiruch; Leanpub*
- https://reactjs.org/docs/hooks-reference.html

Assignment 1

- Assignment in-lab workshops:
  - Requirements Gathering Workshop 1 of 2 (Week 4 prac)
  - Requirements Gathering Workshop 2 of 2 (Week 5 prac)
  - Test and feedback workshop (Week 6 prac)
- Due at the very end of Week 6.
- Please attend the pracs of the group tutor that you are registered in. The tutor = the product owner = your assessor
- Based on exercises covered in week(s) 1-4
  you should be able to get started and complete till the CREDIT part.

RMIT

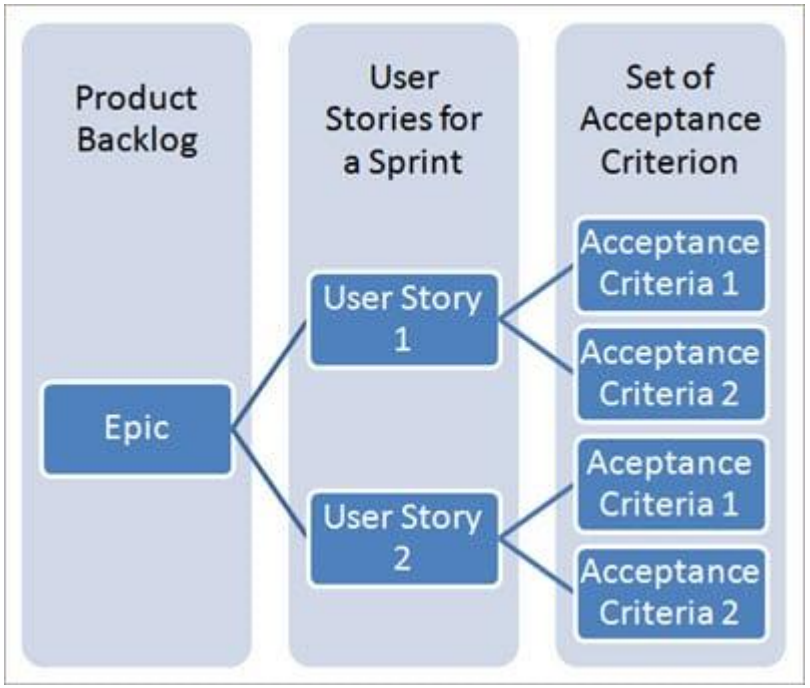
Assignment 1 approach

A suggested approach for every feature ‘epic’:

- Extract the general requirement from the written Assignment specification
- **User Stories** (given in pracs)
- **Acceptance Criteria** (given in pracs but you need to firm them up)
- Think through about the UX and the UI
- Code
- Test
- Repeat until you are ready to deploy it.

RMIT

Assignment 1 approach



What is User Story and Acceptance Criteria? Example

User Story simplifies a requirement from the end-user perspective in a format that both tech and non-tech people understand.

As a _____ (role / type of user)
I want to ___ (action)
So that ____ (a benefit)

RMIT

Assignment 1 approach

User Story

Title:	Priority:	Estimate:
User Story: As a [description of user], I want [functionality] so that [benefit].		
Acceptance Criteria: Given [how things begin] When [action taken] Then [outcome of taking action]		

ProductPlan

User Story Examples in Product Development | Definition and Template

Acceptance criteria define the specific conditions a software feature must meet to be accepted by users.

From each acceptance criterion you can extract:

1. Expected verifiable (test / fail) parameters
2. Positive (expected behaviour) and negative (error handling) scenarios
3. Test cases to ensures these parameters are met

Work through this with your team and ask for feedback from your tutor

RMIT

Assignment 1 approach

User Story: As a user, I want to log in, so that I can use the app.
Acceptance Criteria (AC):

User can log in with valid credentials.
User sees an error message with invalid credentials.

Test Cases (from AC):
TC01: Valid login - Verify user lands on dashboard.
TC02: Invalid password - Verify error message "Incorrect password" displays.

How to Write Test Cases from a User Story - testRigor AI-Based Automated Testing Tool

RMIT

Next week

- Hooks *yet* again
  - useRef
  - useMemo
- Custom hooks *again*
- Interaction between components *again*
- Testing framework
- Lab: A1 Requirements Gathering Workshop 2 of 2